

ProflQ: Professor Recommendation System

Final Project Report - CS 210 Data Management for Data Science

Student: Afnan Haider

Date: May 2026

Repository: <https://github.com/Afn377/ProflQ>

Demo video: <https://youtu.be/o1A1l2W680g?si=4a1GuE1157LvYent>

Project type: End-to-end data management, NLP analytics, and web application

Executive summary

ProflQ is a web app that helps students search for professors, compare them, and understand review patterns in one place. It combines a Django backend, a React frontend, public review data, sentiment analysis, and a normalized database. A big focus of the project is data management: cleaning records, avoiding duplicates, keeping sources organized, making the API reproducible, and not storing more raw review text than needed.

- Uses a normalized professor-review database with separate tables for sources, departments, professors, courses, reviews, sentiment results, and professor stats.
- Builds a large professor catalog without saving every scraped review permanently.
- Compares a tuned VADER baseline with TF-IDF + Logistic Regression and DistilBERT.
- Uses MiniLM embeddings to find professors with similar review language.
- Includes a working web interface for search, filters, professor pages, live reviews, charts, and comparisons.

1. Problem Definition and Background

Students usually choose classes by checking several different places: the course catalog, advice from friends, RateMyProfessors ratings, and posts online. Each source can be useful, but the information is scattered and hard to compare. ProflQ tries to bring those signals together. It stores professor and course information in a clean database, computes review-based analytics, and makes the results available through an API and a web interface.

This connects closely to CS 210 because the main challenge is not just building a website. The project involves schema design, data cleaning, deduplication, ETL commands, source integration, and model evaluation. I also had to make a storage tradeoff. The app keeps professor metadata and aggregate stats in the database, but raw review text is usually fetched live or processed in limited analysis jobs. That keeps the database smaller and avoids keeping unnecessary copies of scraped text.

Prior work and tools

For sentiment, I started with VADER because it is simple, explainable, and designed for short social text [1]. I also compared it with DistilBERT, which is a smaller version of BERT [2, 3]. For similar-professor search, I used sentence embeddings based on the same general idea as Sentence-BERT semantic similarity [4]. The project is not just about using these tools separately; the main work is fitting them into a full data pipeline for professor search and comparison.

Research questions

- Can public professor review signals be organized into a clean relational system that supports fast search, comparison, and analytics?
- Does a trained sentiment classifier improve over a hand-tuned VADER baseline on labeled RMP review text?
- Can content embeddings identify professors with similar teaching-style language better than random matching?

2. Data Description

ProfIQ uses three main data layers. The first is a small seed JSON file, which is useful for local testing and demos. The second is the RateMyProfessors directory crawl, which fills the catalog with professor names, schools, departments, external IDs, average ratings, and rating counts. The third is a separate ML corpus in Parquet format. That corpus contains review text for training and evaluation, but it is not treated as the normal production database.

Dataset / source	Format	Fields used	Role in project
Seed reviews	JSON	source, department, course, professor, review text, rating	Small reproducible demo path for local setup and ORM/API checks.
RMP catalog	SQLite rows from crawler	professor name, institution, department, external_ref, average rating, rating count	Large searchable catalog; current local DB contains 1,702,324 professors and 3,272 departments.
Live RMP reviews	API response, not persisted	comment, helpfulness/clarity rating, course, date	Detail-page review display and lazy aggregate sentiment analysis.
Reddit mentions	Public JSON/PRAW-compatible collector	comment text, URL, timestamp, professor mention slice	Auxiliary qualitative signal on first live-review page.
ML corpus	Parquet	350,018 review rows across 4,793 professors and 49 institutions	Training/evaluating classifiers and building recommender embeddings.

3. Database and Pipeline Methodology

The database schema separates information that would otherwise get repeated inside raw review records. A professor can belong to a department and can be connected to many courses. A review links back to one professor, one source, and optionally one course. SentimentResult stores the analysis for a saved review, while ProfessorStats stores the summary numbers used by the dashboard. I also added uniqueness rules for professor identity, external IDs, course codes, source names, and review URLs so repeated ingest commands do not create duplicate rows.

Database tables and datatypes

Table	Field	Datatype	Purpose / notes
Source	id	BigAutoField / integer primary key	Django-generated identifier.
Source	name	CharField(64), unique	Review platform name such as RateMyProfessors or Reddit.
Source	base_url	URLField / varchar(200), blank allowed	Base website URL for the source.
Department	id	BigAutoField / integer primary key	Django-generated identifier.
Department	name	CharField(128), unique	Department name used for filtering and grouping.
Department	code	CharField(16), blank allowed	Optional short department code.
Professor	id	BigAutoField / integer primary key	Django-generated identifier.
Professor	name	CharField(128), indexed	Professor display/search name.
Professor	department_id	ForeignKey to Department, nullable	Optional department relationship; SET_NULL on delete.
Professor	institution	CharField(128), indexed, blank allowed	School or institution name.
Professor	bio	TextField, blank allowed	Optional free-text biography field.
Professor	external_ref	CharField(64), indexed, unique when nonblank	External source id such as rmp:12345.
Professor	source_avg_rating	FloatField, nullable	Profile-level rating from the source crawl.

Table	Field	Datatype	Purpose / notes
Professor	source_num_ratings	IntegerField	Profile-level rating count from the source crawl.
Professor	created_at	DateTimeField	Set automatically when the row is created.
Course	id	BigAutoField / integer primary key	Django-generated identifier.
Course	code	CharField(32), indexed, unique	Course code such as CS101.
Course	title	CharField(200), blank allowed	Optional course title.
Course	department_id	ForeignKey to Department, nullable	Optional department relationship; SET_NULL on delete.
Course.professors join table	id	BigAutoField / integer primary key	Django-created table for the Course-Professor many-to-many relationship.
Course.professors join table	course_id	ForeignKey to Course	One side of the many-to-many relationship.
Course.professors join table	professor_id	ForeignKey to Professor	Other side of the many-to-many relationship.
Review	id	BigAutoField / integer primary key	Django-generated identifier.
Review	professor_id	ForeignKey to Professor	Required professor; CASCADE on delete.
Review	course_id	ForeignKey to Course, nullable	Optional course; SET_NULL on delete.
Review	source_id	ForeignKey to Source	Required source; PROTECT on delete.
Review	text	TextField	Persisted review text for seed/demo data.
Review	rating	FloatField, nullable	Optional 1-5 rating; Reddit comments usually have none.
Review	source_url	URLField / varchar(200), blank allowed	Unique with source when nonblank to avoid duplicate imports.
Review	posted_at	DateTimeField, nullable	Original review timestamp when available.
Review	ingested_at	DateTimeField	Set automatically during ingest.
SentimentResult	id	BigAutoField / integer primary key	Django-generated identifier.
SentimentResult	review_id	OneToOneField to Review	One sentiment row per persisted review; CASCADE on delete.
SentimentResult	compound	FloatField	Final sentiment score in the VADER-style -1 to 1 range.
SentimentResult	positive / neutral / negative	FloatField	Raw VADER distribution components.
SentimentResult	label	CharField(10)	positive, neutral, or negative.
SentimentResult	themes	JSONField list	Detected theme names for the review.
SentimentResult	analyzed_at	DateTimeField	Updated automatically when analysis is saved.
ProfessorStats	id	BigAutoField / integer primary key	Django-generated identifier.
ProfessorStats	professor_id	OneToOneField to Professor	One aggregate dashboard row per professor; CASCADE on delete.
ProfessorStats	review_count	IntegerField	Number of analyzed reviews/comments.
ProfessorStats	avg_compound	FloatField	Mean compound sentiment score.

Table	Field	Datatype	Purpose / notes
ProfessorStats	positive_count / neutral_count / negative_count	IntegerField	Sentiment label counts.
ProfessorStats	theme_counts	JSONField object	Map from theme name to frequency.
ProfessorStats	recommendation_score	FloatField	Computed 0-100 recommendation score.
ProfessorStats	updated_at	DateTimeField	Updated automatically when stats are saved.

Stage	Persistent writes	Intentionally not written	Reason
Directory crawl	Professor, Department, source rating/count	Raw review text	Keeps the catalog searchable without storing millions of review rows.
Seed ingest	Sources, courses, demo reviews, sentiment rows, stats	Duplicate source URLs	Provides a reproducible small-data path for setup and tests.
Lazy analyze	One ProfessorStats row per professor	Review/comment text	Computes aggregates once while respecting the minimal-storage design.
Live review endpoint	Nothing persistent	Any database rows	Returns per-review sentiment inline for the user session.

This design choice is important. In the larger SQLite database, the Review and SentimentResult tables may be empty after a directory crawl, and that is expected. The big catalog focuses on professor metadata and summary ratings. When the app needs review text, it fetches a bounded amount, analyzes it, and does not keep every raw comment forever. The seed data path still shows the full review schema for testing and demonstration.

4. Sentiment and Recommendation Methods

Hypothesis

My main ML hypothesis is that the wording of a review is enough to predict whether the review is negative, neutral, or positive. I expected a trained classifier to do better than VADER on held-out RateMyProfessors reviews because it can learn patterns from this specific domain. For recommendations, my hypothesis was that professors with similar comments about clarity, workload, grading, and helpfulness should end up close together in embedding space.

Training data and target variable

The supervised sentiment models are trained on the ML corpus built from RateMyProfessors review pages. Each row is one review. It includes the professor's external ID, name, school, department, course, review text, rating, difficulty, date, source URL, and the would-take-again field when it is available. For classification, the input is the review text. The target variable is a three-class sentiment label created from the 1-5 rating: ratings ≤ 2.0 are negative, ratings above 2.0 and up to 3.5 are neutral, and ratings above 3.5 are positive.

I used the source URL to make stable train, validation, and test splits. The URL is hashed, so the same review always goes into the same split, even if the script is run again. That makes the comparison between VADER, TF-IDF + Logistic Regression, and DistilBERT fairer.

Sentiment pipeline

The baseline sentiment model starts with NLTK VADER, then adds rules for professor reviews. I added review-specific words like 'avoid', 'useless', 'helpful', and 'engaging'. I also added phrase rules for comments like 'avoid like the plague' and 'would not recommend', because VADER can miss the meaning of those phrases. When a star rating is available, the analyzer blends it with the text score. The same pass also tags themes such as clarity, fairness, workload, helpfulness, engagement, and grading.

The learned pipeline has two paths. The model used by the app is TF-IDF + Logistic Regression. It lowercases review text, turns it into unigram and bigram TF-IDF features, and trains a class-weighted logistic regression classifier. I chose it for the live app because it is small and fast. I also fine-tuned DistilBERT on the same labels as a stronger comparison model, but it is larger and slower, so it makes more sense as an offline evaluation model.

Recommendation pipeline

For similar-professor search, I group reviews by professor and combine a limited amount of text for each person. That text is encoded with the all-MiniLM-L6-v2 sentence-transformer model. The vectors are normalized and saved as an embedding matrix. When the app needs similar professors, it compares vectors with cosine similarity, loads the matching professor rows from the database, and tries to show same-department or same-school matches first.

Results

The results mostly support the sentiment hypothesis. Both learned models perform better than the VADER baseline on held-out reviews. VADER is still useful because it is simple and always available, but it has trouble with neutral reviews and some professor-specific wording. TF-IDF + Logistic Regression has the strongest macro-F1 in my saved results, which matters because macro-F1 gives more weight to the smaller neutral class. DistilBERT has the highest accuracy and weighted-F1, but it is not as practical for the live app. The recommender results also support the embedding idea: nearest-neighbor matches share departments and institutions more often than a random baseline, so the review embeddings are capturing teaching-style similarity better than chance.

5. API and Frontend Implementation

The backend provides REST endpoints for the main user actions. The React/Vite frontend calls those endpoints to show search results, professor pages, comparison views, sentiment charts, theme charts, live reviews, and similar-professor recommendations.

Endpoint	Purpose
GET /api/summary/	Platform summary and top-ranked professors.
GET /api/professors/	Search/filter by professor, department, institution, course, and sort mode.
POST /api/professors/	Validated, throttled, deduplicated student-submitted professor creation.
GET /api/professors/<id>/	Professor detail with courses, stats, source ratings, and lazy-analysis trigger.
GET /api/professors/<id>/reviews/	Live RMP review page plus first-page Reddit mentions, analyzed inline.
GET /api/compare/?ids=...	Compact multi-professor comparison data.
GET /api/professors/<id>/similar/	MiniLM-based similar-professor recommendations when embeddings are available.

6. Results and Analysis

I checked the implementation with backend tests and a frontend production build. The sentiment tests cover the added lexicon, phrase rules, rating blending, ML fallbacks, and recommender warm-up behavior. The professor creation tests cover validation, duplicate handling, fictional or joke-name rejection, and normal successful submissions.

Check	Result	Interpretation
python3 -m unittest sentiment.tests -v	28 passed	Core sentiment/recommender helpers behave as expected.
python3 manage.py test professors -v 2	13 passed	Create-professor API validation/dedup logic passes.
npm run build	Passed	Frontend compiles successfully; Vite reports only a non-fatal chunk-size warning.

The model comparison shows that the learned models do better than VADER on the main metrics. DistilBERT has the highest accuracy and weighted-F1, while Logistic Regression has the strongest macro-F1. That is important because neutral reviews are common in real course selection. If the model almost never predicts neutral, the app can make

student opinion look more extreme than it really is.

Model	Test rows	Accuracy	Macro-F1	Weighted-F1	Use
VADER + domain rules	7,427	0.764	0.533	0.737	Always-available baseline.
TF-IDF + LogReg	52,445	0.802	0.719	0.817	Live default ML augmentation.
DistilBERT	7,427	0.857	0.636	0.831	Offline comparison / optional live model.

Figure 1. Sentiment model metrics from the saved evaluation artifacts.

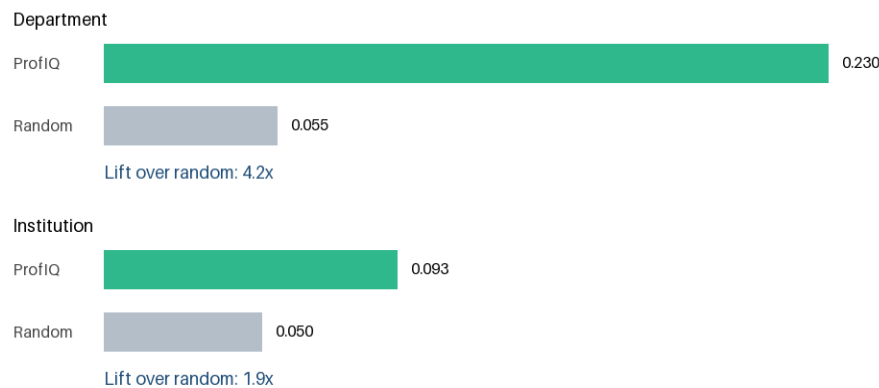


For the recommender, I used purity@5, which checks how many of a professor's top five neighbors share the same department or institution. Department purity is 0.230 compared with a 0.055 random baseline, which is a 4.2x lift. Institution purity is 0.093 compared with 0.050 random, a 1.9x lift. The stronger department result is a good sign because it suggests the embeddings are learning subject and teaching-language patterns, not just school names.

Metric	ProfIQ purity@5	Random baseline	Lift	Interpretation
Department	0.230	0.055	4.2x	Embeddings recover teaching/content similarity better than chance.
Institution	0.093	0.050	1.9x	Location signal exists but is weaker than department/content signal.

Figure 2. Similar-professor recommender lift over random baselines.

Content-based recommender purity@5 vs. random baseline



7. Discussion, Limitations, and Ethics

The main strength of ProfIQ is that it covers the full data lifecycle. It collects data, cleans it, normalizes it, removes duplicates, computes analytics, serves an API, visualizes results, and evaluates models. The app is usable on its own, but the most important data-management idea is the boundary between data that should be stored permanently and raw review text that should only be processed temporarily.

- Source bias: RMP and Reddit comments are self-selected, so they may overrepresent very positive or very negative experiences.
- Label noise: RMP ratings are useful labels, but they do not always match the actual wording of a review.
- Neutral-class difficulty: neutral reviews are hard for the models, even though Logistic Regression improves macro-F1.
- External-source reliability: RMP and Reddit can rate-limit requests or change their pages, so the app needs to fail gracefully.
- Ethics and terms: the project limits raw-text storage and is meant for educational analysis, not redistributing scraped review datasets.
- Operational scaling: a production version should use PostgreSQL and a real background queue instead of SQLite and in-process threads.

Future work could improve entity matching across campuses, add better topic/theme extraction, calibrate sentiment scores, let users adjust their preferences, improve cache invalidation, and continue polishing frontend accessibility and performance.

8. Reproducibility

The repository includes the backend, frontend, migrations, requirements, evaluation notebook, report files, and example data. A fresh local run can use the seed ingest command. Larger crawls and ML artifacts can also be rebuilt with the management commands described in the README.

Demo video: <https://youtu.be/o1A1I2W680g?si=4a1GuE1157LvYent>

Step	Command
Install backend	<code>python3 -m venv .venv; source .venv/bin/activate; pip install -r backend/requirements.txt</code>
Initialize database	<code>cd backend; python manage.py migrate; python manage.py ingest_seed --reset</code>
Run backend	<code>python manage.py runserver 8000</code>
Run frontend	<code>cd frontend; npm install; npm run dev</code>
Run tests	<code>cd backend; python3 -m unittest sentiment.tests -v; python3 manage.py test professors -v 2</code>
Build frontend	<code>cd frontend; npm run build</code>

9. Conclusion

Overall, ProfiQ meets the project goal by combining a working web app with a documented data-management workflow. It brings together public review signals, normalizes professor and review data, computes sentiment and theme summaries, compares learned models with a rule-based baseline, and shows the results in a usable interface. The final design is practical: the database stays searchable and small, but students can still open a professor page and see review-level evidence when they need it.

References

- Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *International AAAI Conference on Web and Social Media*.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*.
- Django Software Foundation. Django documentation. <https://docs.djangoproject.com/>
- React documentation. <https://react.dev/>

AI assistance disclosure: I used ChatGPT for help with debugging, explanations, editing, and organizing the project. I did not use it to replace my own work or submit generated content as the project itself.